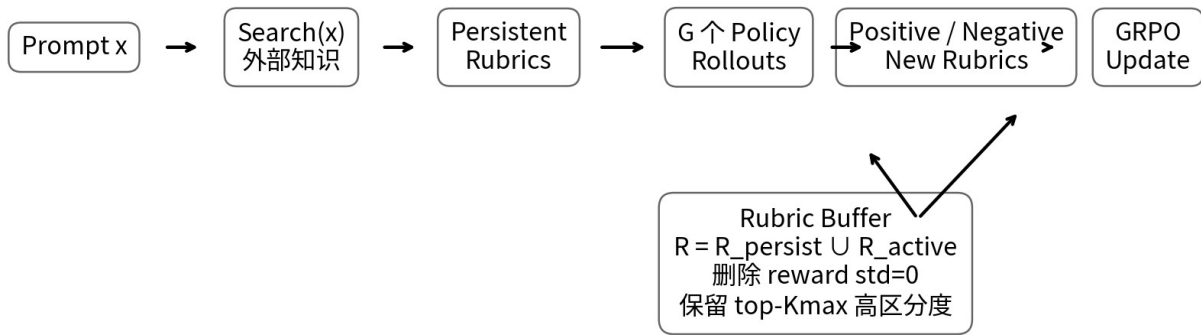


DR Tulu

Reinforcement Learning with Evolving Rubrics for Deep Research

完整技术研究 · Know-Why × Know-How × 源码/训练/实验



核心：奖励函数不再静态；Verifier 的“知识边界”和“判分标准”随 Policy 一起更新

核心机制概览：Reward / Verifier 与 Research Policy 协同演化

研究版本：论文 v3（2026-05-15） / ICML 2026 + 官方代码、模型与数据状态

整理日期：2026-08-13

阅读导航与核心结论

一句话结论 DR Tulu 的真正突破不是“更强的 Search Agent”，而是把长篇 Deep Research 的奖励设计从静态 Judge 提升为一个会随 Policy 探索而更新的、搜索增强的 Verifier 系统。

如果把 DeepResearcher 的核心问题写成 “Where should RL happen? — on the real web”，那么 DR Tulu 的核心问题就是 “What should long-form research optimize? — evolving, evidence-grounded rubrics”。这使长篇、多来源、带引用的研究报告第一次可以用端到端 RL 直接优化，而不必退回到手写固定 workflow。[S1][S2]



演进焦点：Where to search → How to act → What to reward

图 1 | Deep Research RL 的问题迁移：环境 → 策略 → 奖励

本报告重点回答的 10 个问题

- 为什么短答案 RLVR 无法自然扩展到长篇 Deep Research?
- RLER 为什么要让 Rubric 与 Policy 一起演化，而不是固定一次生成?
- Search-based Rubrics、Positive Rubrics、Negative Rubrics 分别解决什么问题?
- 为什么用 reward variance / std 管理 Rubric Buffer 是关键工程选择?
- SFT 冷启动为何仍然必要? 哪些数据建立了最初的 Tool/Citation 行为?
- 在线 GRPO 如何与真实工具调用、异步工具请求、缓存和限流结合?
- MCP / dr-agent-lib 在训练系统里承担的是“工具协议”还是“训练 Runtime”角色?
- 实验中哪些增益来自 RLER，哪些只是 Citation/Format 辅助奖励?
- 为什么 8B 模型能在长篇 Research Benchmark 上逼近闭源 Deep Research?
- 这些机制如何迁移到自己的 Research Agent / Coding Agent / Enterprise Agent 训练体系?

1. 时代背景：Deep Research 的瓶颈从 Search 转向 Reward

2024–2025 年的开放 Deep Research 主要沿两条路线发展：一条是 inference-time workflow，例如 STORM、GPT Researcher、Open Deep Research；另一条是把搜索能力训练进模型，例如 Search-R1、DeepResearcher、

WebThinker、Tongyi DeepResearch。前者受限于是手写控制流，后者大量依赖短答案 QA 的 RLVR——因为答案可以 exact match、F1 或规则验证，奖励干净。

问题在于，真实研究报告并不存在唯一正确字符串。一个高质量回答需要同时满足：信息覆盖、事实正确、证据相关、跨来源综合、引用可追溯、结构清晰、深度与篇幅适当。任何单一标量 Judge 都很容易把这些维度压扁，并产生 reward hacking。论文明确把这一点作为 RLER 的出发点。[S1]

Know-Why 长篇生成不是“答案空间更大一点”，而是评价函数本身不完备。只扩大 rollout、增加搜索次数，无法解决 objective misspecification。

1.1 从 Verifiable Reward 到 Verifier System

短答案 RLVR 的理想化形式是 $\text{Reward} = \text{Verify}(\text{final_answer}, \text{gold})$ 。而 Deep Research 更接近 $\text{Reward} = \text{Verify}(\text{claims}, \text{evidence}, \text{coverage}, \text{synthesis}, \text{instructions}, \text{style}, \text{citations}, \text{task-specific expectations})$ 。DR Tulu 的关键转变是：不再假设这一组标准可以一次写完，而是把标准本身作为训练状态。

因此可把 RLER 看成二层学习系统：内层 Policy 学“如何研究”；外层 Rubric System 学“当前什么行为值得奖励、什么新 failure mode 需要惩罚”。

2. Problem Formulation：Research Agent 的 Action Space

论文把 DR 模型定义为配备搜索相关工具的语言模型，动作空间为 {think, tool, answer, cite}。tool 触发环境执行并返回 observation；think / cite / answer 直接进入上下文；当模型选择 answer 时终止。[S1]



Action space = {think, tool, answer, cite}；停止条件由模型选择 answer

图 2 | DR Tulu 推理时的自主搜索循环

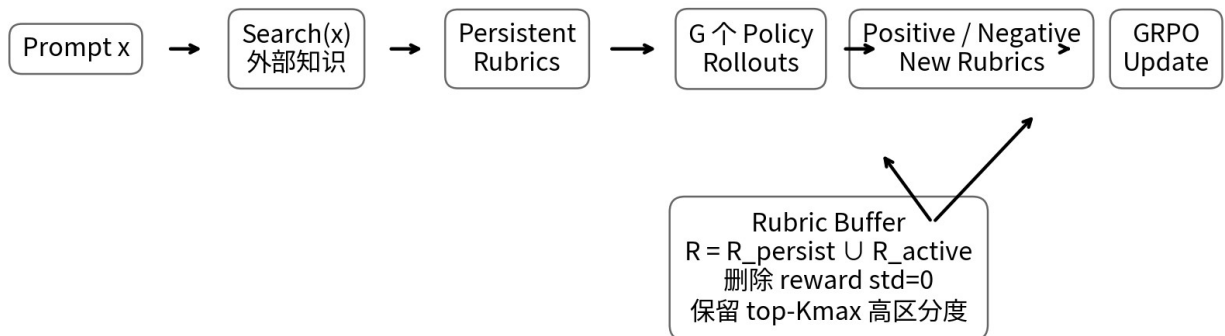
2.1 为什么 cite 被视为一等动作

很多 Deep Research 系统把引用放到写作后处理阶段。DR Tulu 反过来要求模型在生成 claim 时绑定 snippet ID，使 provenance 进入 Policy 的输出协议。这样训练时 Citation Reward、最终评测时 Citation Precision/Recall，以及用户验证时 Claim→Evidence 映射可以使用同一条数据链。

Know-How Citation 不应只是 renderer feature。最稳妥的设计是让检索结果从产生时就拥有稳定 Evidence ID，并要求 Policy 在 claim 层引用该 ID。

3. RLER：让 Reward Function 与 Policy Co-evolve

RLER = Reinforcement Learning with Evolving Rubrics。每个训练问题 x 都维护一个 Rubric Buffer： $R_x = R_{\text{persist}} \cup R_{\text{active}}$ 。训练前用搜索结果生成 Persistent Rubrics；训练过程中读取当前 Policy 的多条 rollout，再生成新的 Positive / Negative Rubrics；最后基于 Rubric 对 rollout 打分并执行 GRPO。[S1]



核心：奖励函数不再静态；Verifier 的“知识边界”和“判分标准”随 Policy 一起更新

图 3 | RLER 的核心闭环：Search-grounded rubric + rollout-contrastive rubric + GRPO

3.1 Persistent Rubrics：把外部知识注入 Verifier

对于每个训练 prompt，系统先执行 SEARCH(x)，把问题与检索文档交给 Rubric Generator。其目的不是给 Policy 更多上下文，而是给 Verifier 更多 “privileged information”，形成 generation-verification gap：Policy 必须自己探索；Verifier 可以利用额外搜索证据判断它做得是否正确。

这是一个非常值得迁移的思想：训练 Agent 时，Actor 与 Verifier 不必拥有完全相同的信息预算。Verifier 可以访问更多证据、更强检索、更慢模型，只要这些 privileged signals 不泄露到推理时。

3.2 Evolving Rubrics：把当前 Policy 的行为变成新的评价维度

每轮为同一 prompt 采样 G 个 rollout。Rubric Generator 同时看到这些 response、搜索轨迹以及现有 Rubric Pool，然后寻找“当前 rubric 没有覆盖、但能区分好坏”的新标准。Positive rubric 捕获新出现的高价值证据/策略；Negative rubric 捕获 reward hacking、冗余、复制网页、无关代码、滥用 citation 等失败模式。[S1]

这不是普通 self-critique：新 rubric 会进入后续训练，真正改变 reward landscape。换言之，探索产生新行为，新行为反过来扩展 evaluator 的判别边界。

3.3 Rubric Buffer：为什么用 reward variance 做保留准则

如果每轮都增加 rubric，训练成本会线性增长。DR Tulu 对每个 rubric 计算它在当前 G 条 rollout 上的 reward 方差/标准差：std=0 表示所有 rollout 都一样满足或一样失败，当前没有区分价值，直接删除；其余按 std 排序，只保留 top-Kmax。最终训练设置 Kmax=5。[S1]

Know-Why Rubric 的价值不是“是否正确”，而是“是否能在当前 Policy 分布上提供学习信号”。这本质上是 active evaluator / curriculum selection。

4. Rubric 质量：Search-grounded 为什么比 Closed-book 更好

论文把 rubric 分成 General、Closed-book instance-specific、Initial Search-based、Evolving。搜索增强 rubric 更容易写成“具体可断言、可核验”的标准，而不是“讨论 X”“全面说明 Y”这类宽泛描述。论文 Table 6 显示 Initial/Evolving rubrics 的 assertive fraction 超过 0.5，而 closed-book 只有 0.22；factuality 也接近 1。[S1]

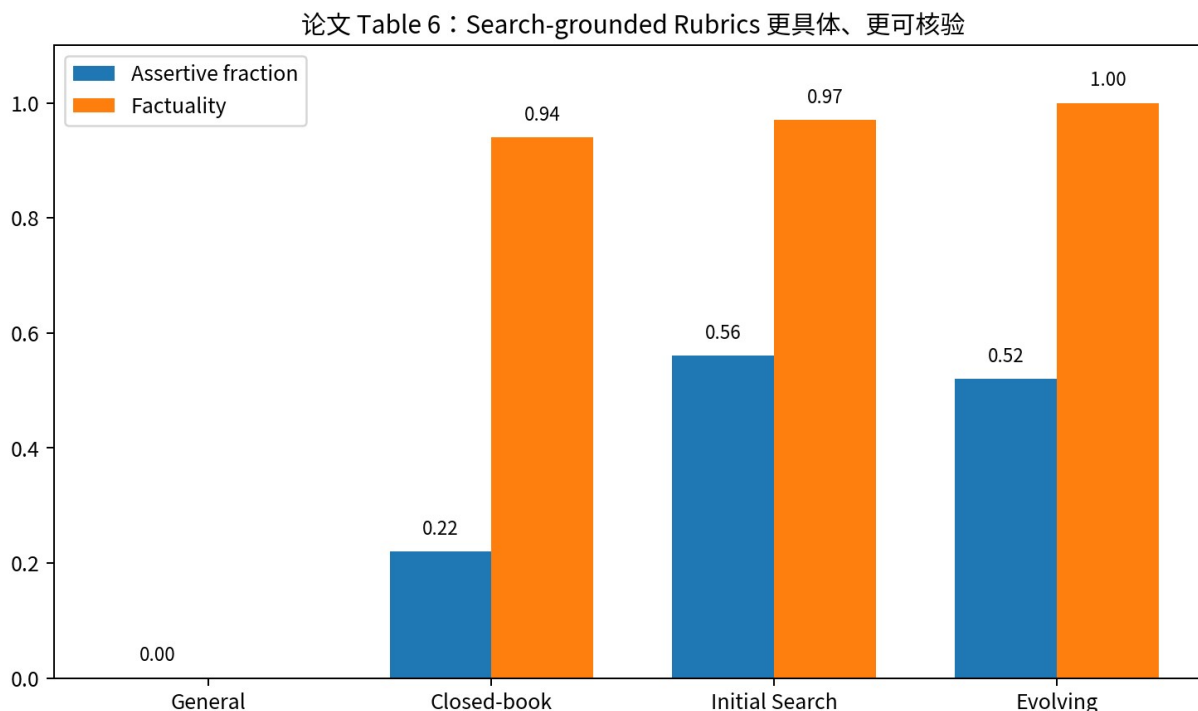


图 4 | Search-grounded Rubrics 的具体性与事实性

4.1 General Judge 为什么容易被 Hack

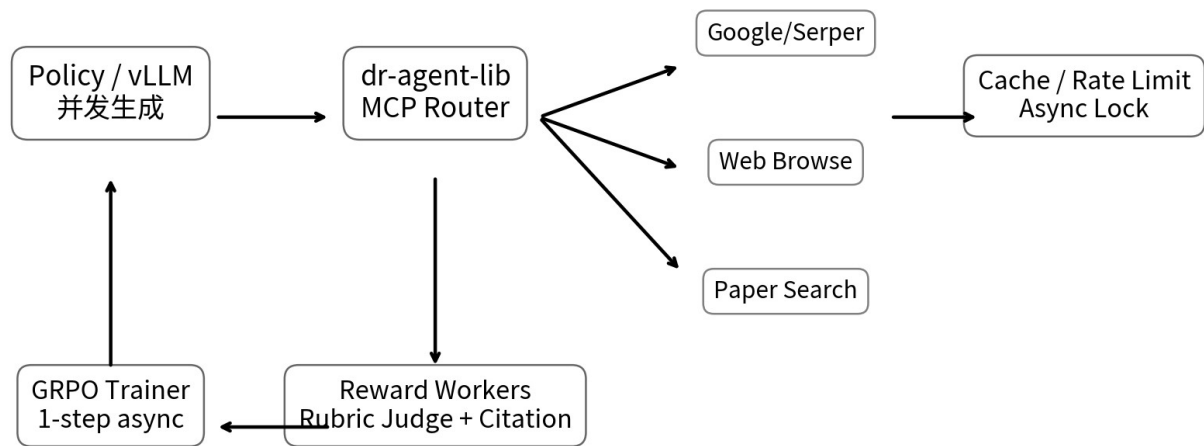
单一 general rubric（如 “comprehensive / thorough / factual / coherent”）看起来合理，但它没有回答“对这个具体问题，哪些事实必须出现、哪些证据最关键”。Policy 很容易学会篇幅、结构、措辞等 Judge 偏好，而不是学会研究。论文消融中 general rubric 甚至低于 SFT baseline。[S1]

4.2 Negative Rubric 是一个在线 Anti-Reward-Hacking 机制

论文 toy study 中出现了意外 Python code 行为。初始 static rubric 无法预见它；evolving negative rubric 会在多条 rollout 对比中识别这类共同失败并加入惩罚，随后这种行为频率下降。这说明 RLER 的一个重要能力是把“新发现的 anti-pattern”自动写入训练 objective。

5. dr-agent-lib：MCP 在这里不是插件层，而是 RL Runtime

论文与官方仓库将 dr-agent-lib 作为完整开放栈的一部分。它包含三个核心能力：统一 MCP tool backend；高并发异步 request 管理、全局 cache 与 process lock；可组合 prompt/workflow 层。训练和评测使用的默认工具覆盖 general web search、web browse、paper/snippet search。[S1][S3]



系统瓶颈往往不在 GPU，而在外部 API latency / rate limit；异步 tool calling 是训练吞吐的关键 runtime primitive

图 5 | 异步 Tool Runtime：训练吞吐由 Tool latency / rate limit 强烈决定

5.1 Tool Plane

类别	论文列出的工具/实现	作用
General Search	Serper Google Search / massive-serve	网页搜索 / dense passage retrieval
Scholar Search	Semantic Scholar / snippet search / PubMed / Google Scholar	论文元信息或段落检索
Browse	Serper fetch / Crawl4AI / Jina	URL → 页面可读文本
Reranker	vLLM hosted reranker	文档重排

推理统一使用三个抽象工具：google_search、web_browse、paper_search；不对不同 benchmark 做 task-specific pipeline customization。SQA_{v2} 会自然偏向 paper search，而通用/健康任务更偏向 web search。[S1]

5.2 为什么异步 Tool Calling 是训练级必要条件

在 rollout 生成时，一条序列触发工具调用后进入 sleep，GPU 继续生成其他 rollout；工具返回后再恢复该序列。这样 generation 与 I/O 重叠。论文最终训练 70 天、约 27,000 GPU 小时，但明确指出加更多 compute 并不能线性加速，因为瓶颈转移到了 API rate limits。[S1]

Know-How 对 Tool-using RL，吞吐模型不能只看 tokens/s。真正要优化的是 GPU decode、网络 I/O、API quota、cache hit、judge calls、rollout scheduling 的联合流水线。

6. SFT Cold Start：RL 之前先建立 Research Prior

直接对基础模型做长篇 RLER 会得到低质量工具使用与报告，探索很难启动。因此 DR Tulu 先做 teacher distillation SFT。教师为 GPT-5，生成显式 mock thinking、tool calls、tool outputs 与最终带 citation 的完整轨迹。[S1][S2]

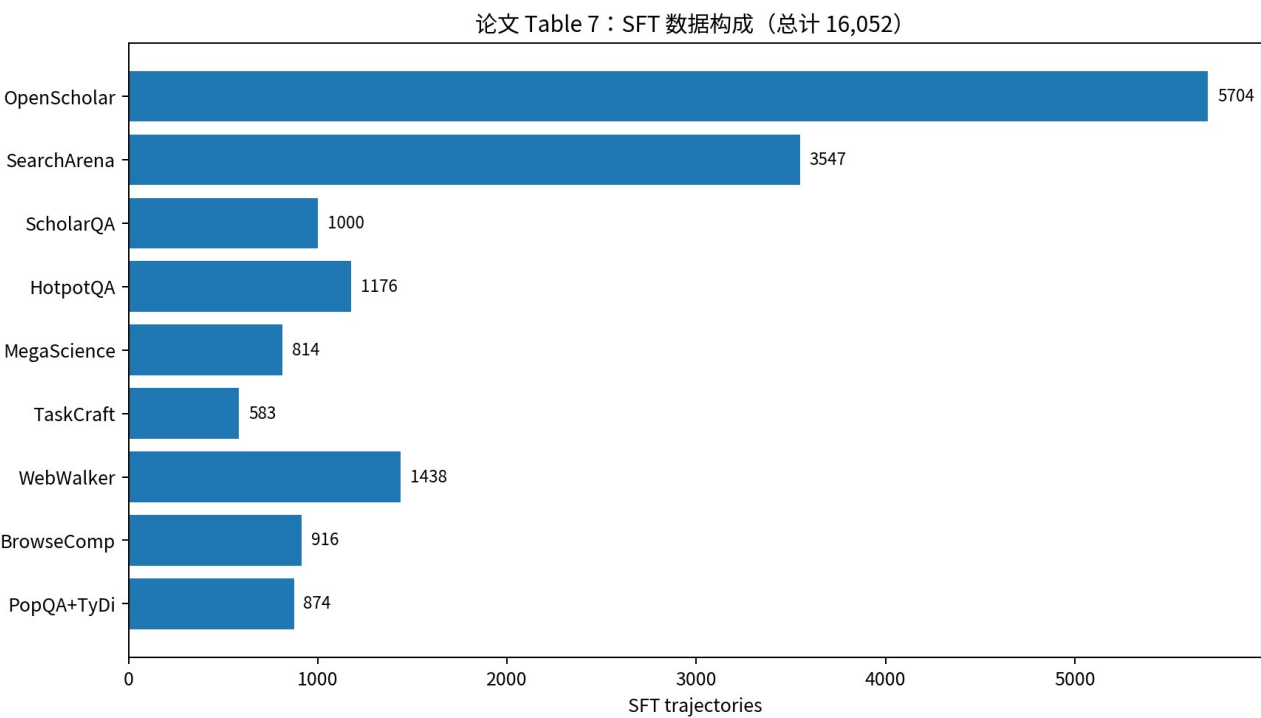


图 6 | 论文 Table 7：16,052 条 SFT trajectory 的来源

6.1 数据混合的设计逻辑

长篇：OpenScholar 5,704、SearchArena 3,547、ScholarQA 1,000；短篇：HotpotQA、MegaScience、TaskCraft、WebWalkerSilver、BrowseComp、PopQA & TyDi QA。总计 16,052 条。长篇负责 research depth / citation / synthesis；短篇保留精确回答、较短 evidence chain 与 instruction following。[S1]

当前 Hugging Face 的 dr-tulu-sft-data 为 13,062 行，并明确说明尚不包含 MegaScience、HotpotQA、ScholarQA 子集。因此“论文训练总量”和“Hub 当前可直接下载量”不能混为一谈。[S5]

6.2 Rejection Sampling

- 所有样本：检查 tool-call / answer / citation 协议是否合法。
- 短答案样本：最终答案必须与 gold 匹配。
- 额外 on-policy SFT 实验：长篇轨迹要求 rubric coverage 与 citation precision 都 > 0.6。

值得注意的是，on-policy SFT 虽能提高 standalone SFT checkpoint，却在后续 RL 中最终不如原始 SFT mixture。这说明“离当前策略更近”并不自动等于“更好的 RL 初始分布”；过度自举可能削弱探索多样性。[S1]

7. Online RL：GRPO + Long-Horizon Tool Use

最终 RL 仅使用 long-form prompts。论文主文描述约 5K 来自 SearchArena/OpenScholar + 4K RaR；当前公开 HF RL 数据为 4,881 行，仅包含 OpenScholar/SearchArena，另需从 RaR-Science 抽 3K、RaR-Medicine 抽 1K 才接近最终训练 mixture。[S1][S6]

7.1 核心超参数

超参数	值
Base / Init	Qwen3-8B / DR Tulu SFT
Unique prompts / batch	32
Rollouts / prompt	8
Max response length	16,384 tokens
Packed sequence max	18,500 tokens
Max tool calls	10
Sampling	temperature=1.0, top-p=1.0
KL coefficient	0.001
Learning rate	5e-7 constant
AdamW betas	(0.9, 0.95)
Kmax evolving rubrics	5
Final run	4,000 steps $\approx$ 14.5 epochs, $\sim$ 27,000 GPU hours

实现还采用 token-level GRPO loss aggregation、sample packing、1-step asynchronous training、tool-output masking。Tool observation 不参与 policy loss，这一点非常重要：环境返回的 token 不是模型 action，不应该被当成要最大似然学习的输出。[S1]

7.2 Citation Reward 的阶段性开关

论文最终运行中 Citation Reward 在 step 650 后关闭，因为其信号已饱和且 API judge 成本高；step 3350–4000 又因 citation 表现下降重新开启。消融显示后半程去掉 citation reward 并不降低总分，说明主要收益来自 rubric reward，而不是辅助引用奖励。[S1]

Know-How Auxiliary reward 最好按“是否仍有边际学习信号”动态启停，而不是永久叠加。训练监控应记录每个 reward head 的均值、方差、饱和度与调用成本。

8. 训练系统中的 SFT → RL 因果关系

SFT 的任务不是达到最终上限，而是把模型带入“能探索”的策略区域：会规划、会选择工具、会读结果、会引用、会终止。RLER 才负责在真实 on-policy 分布上优化“研究质量”。

从工程角度，可以把它理解成：SFT 建立 syntax + basic policy prior；RL 建立 task-adaptive search depth + evidence selection + synthesis quality；RLER 建立 adaptive objective。三者缺一都可能导致训练失败。

9. 实验：DR Tulu 强在哪

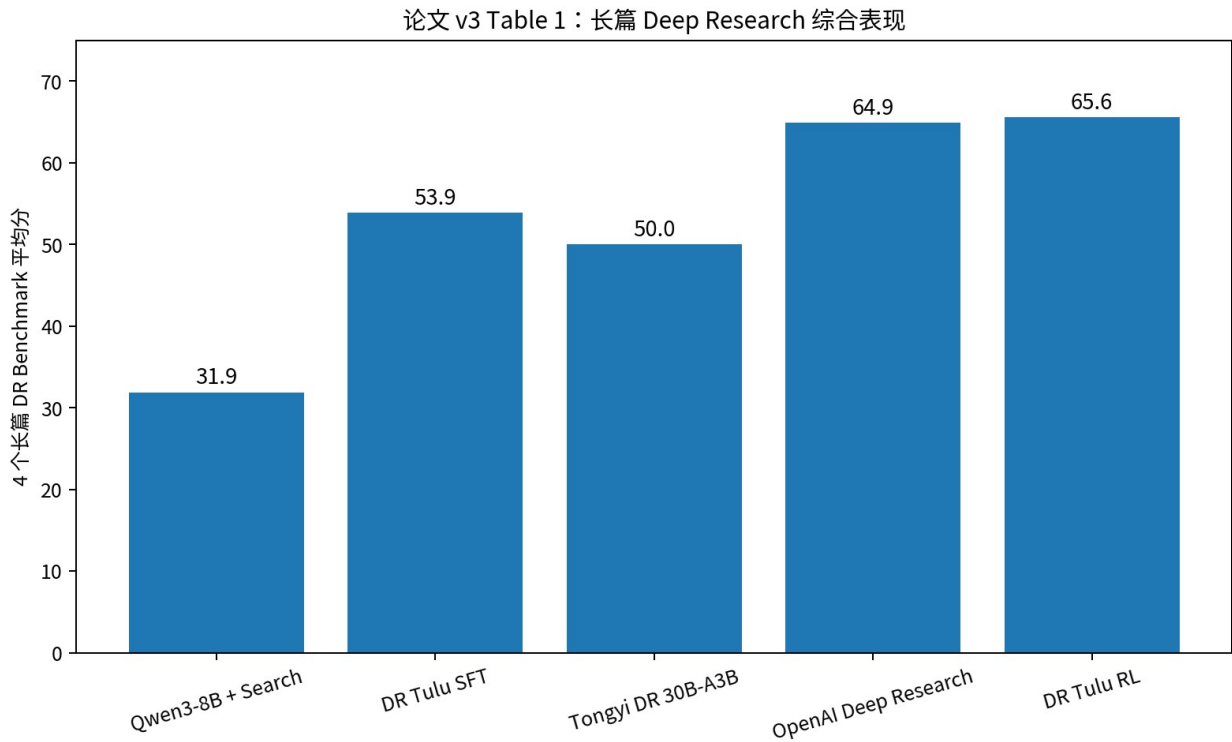


图 7 | 论文 v3 Table 1：4 个长篇 benchmark 平均分

论文 v3 的长篇四榜结果：DR Tulu RL 为 SQA_{v2} 88.3、HealthBench 52.8、ResearchQA 75.7、DRB 45.4，平均 65.6；Tongyi DeepResearch-30B-A3B 平均 50.0；OpenAI Deep Research 平均 64.9（部分 benchmark 为论文引用/子集结果，表内有明确标记）。[S1]

这里最值得关注的不是“8B 击败某个闭源模型”的标题，而是 SFT→RL 的增益：31.9（Qwen3-8B + same search stack）→53.9（SFT）→65.6（RL）。这说明改进不是单纯来自外部搜索基础设施，Policy Training 本身贡献巨大。

9.1 Fine-grained：RLER 改变的是哪些能力

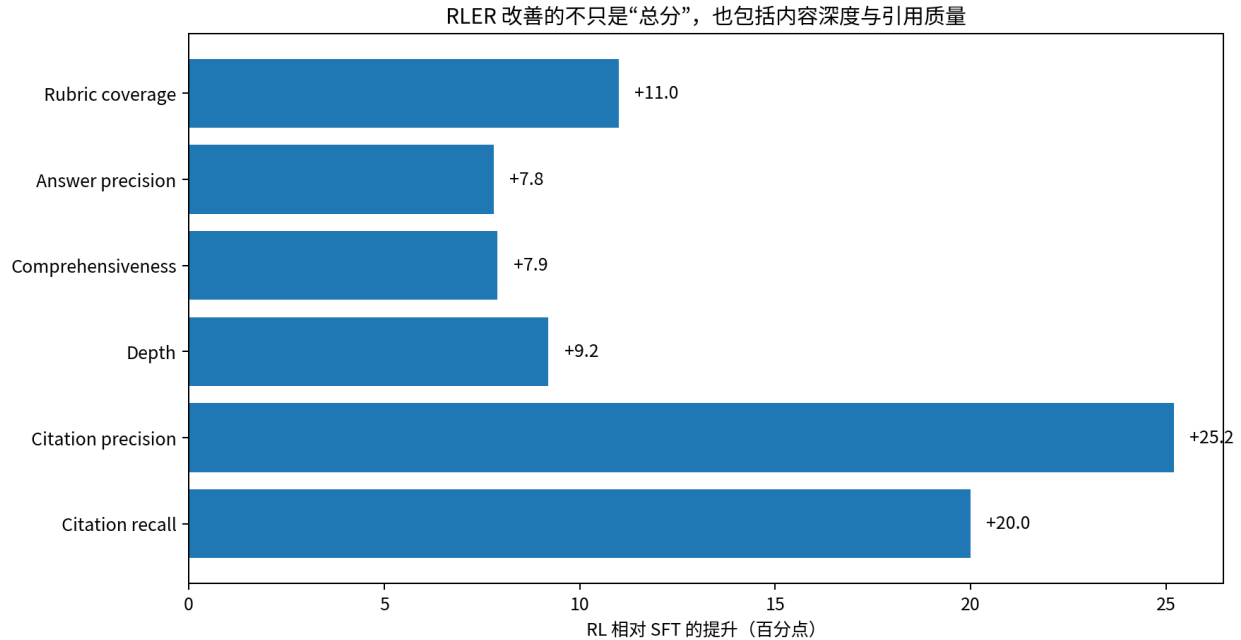


图 8 | 论文 Appendix：RL 相对 SFT 的细粒度增益

论文报告 RL 相对 SFT 在 rubric coverage、answer precision、comprehensiveness、depth、citation precision/recall 上都有显著提升，尤其 citation precision +25.2、recall +20.0。说明 RLER 不只是让答案变长，而是在“内容 + attribution”两个轴上共同提升。[S1]

9.2 成本与行为形态

SQAv2 上，DR Tulu RL 平均答案约 1,889 words、35.8 个 citations、4.3 次 tool calls，论文估算成本约 \$0.0019/query；OpenAI DR 约 6,445 words、79.6 citations、\$1.8/query。论文将 DR Tulu 描述为 Pareto frontier。需要注意：该成本核算按当时公开 token/API 价格估计，且不含自托管硬件资本与运维成本，因此应视为论文定义下的相对比较，而不是完整 TCO。[S1]

10. Ablation：哪些机制真的重要

10.1 Search-based rubric > Closed-book rubric

只用静态 rubrics 时，Search-based rubric 效果最好；通用 rubric 甚至可能低于 SFT。原因非常直接：对知识密集任务，Judge 如果不搜索，就无法知道“哪些具体事实应该检查”。

10.2 Evolving > Initial-only

移除 evolving rubrics 后，平均性能最高下降约 2 点，并且随训练推进差距扩大。这正符合 RLER 假设：Policy 越进化，旧 rubric 越容易失去判别力。[S1]

10.3 不依赖更强闭源 Judge

1000 steps 时，把 GPT-4.1 rubric generator + GPT-4.1-mini judge 全部替换为 Qwen3-8B，平均只低 1.3 点，仍比 SFT 高 4.4 点。这意味着 RLER 的增益主要来自“结构化、搜索增强、on-policy 的评价机制”，而不只是把强模型知识蒸馏进弱模型。[S1]

10.4 Citation Reward 不是主要来源

论文后期 branch ablation：w/ citation reward 平均 60.6，w/o citation reward 61.2。作者据此认为 RLER rubric component 是主要收益来源；format reward 也很早饱和。[S1]

11. Tool Selection：模型开始学习“研究方法”而非固定 Search Recipe

在 SQA_{v2} 这种科学文献任务上，paper search 使用占比约 90%；在 DeepResearchBench 等通用任务上，web search / browsing 占更高比例。论文强调它没有按任务写专门路由，而是统一提供多工具，让 Policy 学会选择。[S1]

Know-Why Research Policy 的一个核心能力是 information-source selection：不仅是“搜什么 query”，还包括“去哪个 evidence universe 搜”。Tool routing 应进入训练，而不是永远写死在 orchestrator。

12. 与 DeepResearcher / WebThinker / Tongyi 的关系

DeepResearcher	WebThinker	Tongyi DeepResearch	DR Tulu
核心问题 真实 Web RL 环境	核心问题 Reasoning-native Search	核心问题 训练通用 Research Po	核心问题 长篇答案如何做 RL
Reward 短答案 F1/结果奖励	Reward DPO / trajectory preference	Reward SFT + RL 多阶段	Reward Search-grounded Evolving Rubrics
输出 偏 QA	输出 QA + Report	输出 Research Agent	输出 带 snippet citation 的长篇报告

关键演进：训练环境 → 工具策略 → Research Policy → Reward / Verifier Co-evolution

图 9 | 四条“训练型 Deep Research”路线的核心问题

系统	主要训练/控制对象	Reward/Verifier	最值得迁移的点
DeepResearcher	Real-web search policy	短答案 outcome / F1	真实环境、长程工具 RL、并发搜索 runtime

系统	主要训练/控制对象	Reward/Verifier	最值得迁移的点
WebThinker	Reasoning + nested web exploration	DPO / preference trajectory	把 Search/Click/Read 纳入 reasoning token stream
Tongyi DeepResearch	通用 Research Agent policy	多阶段 SFT/RL	数据规模化、环境工程、agentic post-training
DR Tulu	Long-form research policy + evaluator	Search-grounded evolving rubrics	Verifier co-evolution、claim citation、long-form RL

可以把 DR Tulu 看成对 DeepResearcher 的自然补全：DeepResearcher 证明“真实 Web 是可训练环境”；DR Tulu 证明“开放长篇任务也可以有足够稳定的在线 reward，只要 evaluator 本身成为一个动态系统”。

13. 源码与开放性：这次“fully open”具体开放了什么

官方仓库分成 agent/、rl/、sft/ 三层：agent 是 dr-agent-lib 与评测；rl 基于 Open-Instruct；sft 基于 LLaMA-Factory。官方同时发布 DR-Tulu-8B、SFT checkpoint、RL/SFT data、评测输出以及 interactive CLI/demo。
[S3][S4][S5][S6]

当前 HF SFT dataset 13,062 rows / 532 MB；RL dataset 4,881 rows / 2.8 MB。RL dataset card 明确指出最终训练还使用额外 RaR Science/Medicine 4K，因此真正复现 final run 必须合并外部数据，而不是只下载一个 parquet。
[S5][S6]

13.1 一个值得警惕的文档细节

论文 Table 9 明确写训练最大 tool calls=10；但 Appendix 对某段 training curve 的解释出现“5 total tool calls”的文字。v3 里两处存在表述不一致。工程复现应优先以公开 config / 当前代码为准，并记录 commit，而不要只照论文叙述手抄超参数。

14. Know-Why：可以沉淀的 8 条原则

- 长篇 Agent 的真正难点往往不是 Policy，而是 Objective Specification。
- Verifier 应拥有 privileged evidence；Actor 与 Verifier 不必信息对称。
- 静态 Rubric 会随着 Policy 分布变化而失效，评价标准需要 on-policy 更新。
- Positive rubric 扩张能力边界，Negative rubric 压制新型 reward hacking；二者缺一不可。
- Rubric 不是越多越好；应按“当前区分度”做 active buffer management。
- Citation provenance 必须在 Retrieval 阶段就成为结构化状态，而不是写作末端补链接。
- Tool-using RL 的瓶颈是 Runtime：I/O、rate limit、cache、judge concurrency 与 GPU 必须联合设计。
- SFT 是探索的 bootstrap，RL 是能力优化；不要把 SFT 终点误当成 RL 最优起点。

15. Know-How：如何实现一个 Mini-RLER Research Agent

- 定义稳定动作协议：think / call_tool / cite / answer，并给每个 evidence 返回稳定 ID。
- 准备 1K-5K 个真正需要多步检索的 long-form prompts；先用强 teacher 生成完整 trajectory 做 SFT。
- 每个 RL prompt 预搜索一次，用“问题 + 搜索证据”生成 5-10 条 persistent rubrics。
- 每轮对同一 prompt 采样 G=4-8 条 rollout；Rubric Generator 比较多条 response，生成 positive / negative rubrics。

- 5. 每条 rubric 独立 Judge，得到 rubric-level rewards；清除 $\text{std} \approx 0$ 的 rubric，active buffer 保留 top-K。
- 6. 奖励 = weighted rubric score + 小型 format/search/citation auxiliary reward；监控每个 reward head 的方差与成本。
- 7. 使用 GRPO/PPO 类算法；mask tool observation token；尽量 sample-pack；对 tool call 做 async suspend/resume。
- 8. 把 Search API / Browse / Paper DB 全放到统一 Tool Plane，配全局 cache、限流、重试、超时和 tracing。
- 9. 评测必须同时测内容质量、citation precision/recall、tool calls、answer length、cost 和 latency。
- 10. 每次训练保存 Policy checkpoint + Rubric buffer snapshot + tool config + evaluator version，保证可追溯。

16. 一个可实现的系统分层

建议把自己的 Deep Research 训练平台拆成五层，而不是把所有逻辑塞进 trainer：

层	职责	关键状态
Policy Layer	生成 think/tool/cite/answer	trajectory、logprobs
Tool Runtime	MCP/HTTP tool 执行、cache、rate limit	observations、evidence IDs
Evidence Layer	来源归一化、provenance、claim mapping	Evidence Store
Verifier Layer	rubric generation / judge / citation verify	Rubric Buffer、reward vector
RL Runtime	rollout scheduling、packing、async update	advantages、policy version

其中真正“DR Tulu 式”的关键是 Verifier Layer 不是固定函数，而是有自己的状态、更新规则与预算。

17. 局限与风险

- 训练代价仍然高：论文 final run 约 27,000 GPU-hours / 70 天，且外部 API rate limit 成为主要瓶颈。
- Rubric Generator / Judge 仍可能有系统偏差；evolving 并不等于无偏，只是更贴近当前 failure modes。
- Search-grounded rubric 依赖检索质量；如果初始搜索漏掉关键证据，Verifier 也可能建立错误标准。
- 在线 Rubric 生成需要额外 LLM 调用，reward 成本本身可能成为规模化瓶颈。
- 复杂长轨迹具有高方差，论文也报告同一系统重复生成会产生显著差异。
- HealthBench 仍有明显 headroom，说明通用 long-form reward 并不能替代领域专业 evaluator。
- 论文不同段落对 tool-call cap 存在轻微设置不一致，复现必须以代码/config + commit 为准。

18. 下一代方向：从 Evolving Rubrics 到 Evolving Verifier

RLER 的自然下一步不是继续堆更多 rubric，而是把 Verifier 进一步模块化：Claim Extractor → Evidence Retriever → Contradiction Checker → Coverage Analyzer → Rubric Composer → Reward Aggregator。这样 Rubric 不再只是自然语言 checklist，而能绑定可执行验证器。

可以进一步加入：跨训练步的 Rubric Memory、自动聚类 failure mode、按不确定性触发额外验证、不同领域专用 verifier、claim-level reward attribution，以及将 reward cost 纳入 curriculum。

推演公式 Next-Gen Deep Research RL = Policy × Evidence Store × Tool Runtime × Evolving Verifier × Cost-aware Scheduler

19. 结论

DR Tulu 的意义在于，它把 Deep Research 从“如何搭一个研究 workflow”进一步推进到“如何直接训练一个能做研究的模型”，并且补上了长篇任务最难的 Reward/Verifier 缺口。它证明：开放式长篇研究不是不能 RL，而是不能继续沿用 short-form RLVR 的静态答案验证范式。

最值得迁移的工程资产不是某个 checkpoint，而是三个组合：Search-grounded instance rubrics、on-policy positive/negative evolving rubrics、以及能承载高并发真实工具调用的 async agent runtime。把这三者拆开来，每一个都能单独迁移到 Coding Agent、Enterprise Agent、Scientific Agent 等长程任务。

附录 A | 关键数字速查

项目	论文 v3 / 当前公开状态
Base model	Qwen3-8B
SFT trajectories	16,052 (论文 Table 7)
HF SFT data	13,062 rows; 部分子集尚未包含
RL prompts	约 5K SearchArena/OpenScholar + 4K RaR (主文)
HF RL data	4,881 rows; 不含 final run 的 4K RaR
SFT compute	8×H100, 5 epochs, 136 GPU-hours
Final RL compute	16×H100 常规设置, 4,000 steps, 约 27,000 GPU-hours / 70 天
Rollout group size	8
Max response	16,384 tokens
Kmax	5 evolving rubrics / prompt
Tool call cap	Table 9: 10
Main long-form avg	65.6
Tongyi DR avg	50.0
OpenAI DR avg	64.9 (表中部分为引用/子集)

附录 B | 主要一手来源

[S1] DR Tulu: Reinforcement Learning with Evolving Rubrics for Deep Research, arXiv v3 / ICML 2026 — <https://arxiv.org/abs/2511.19399>

[S2] Ai2 Blog: DR Tulu: An open, end-to-end training recipe for long-form deep research — <https://allenai.org/blog/dr-tulu>

[S3] Official GitHub: rlresearch/dr-tulu — <https://github.com/rlresearch/dr-tulu>

[S4] Hugging Face Model: rl-research/DR-Tulu-8B — <https://huggingface.co/rl-research/DR-Tulu-8B>

[S5] Hugging Face Dataset: dr-tulu-sft-data — <https://huggingface.co/datasets/rl-research/dr-tulu-sft-data>

[S6] Hugging Face Dataset: dr-tulu-rl-data — <https://huggingface.co/datasets/rl-research/dr-tulu-rl-data>

[S7] AllenAI Open-Instruct — <https://github.com/allenai/open-instruct>

证据边界说明：本报告以论文 v3 (2026-05-15) 为实验主证据；博客用于解释工程动机与发布内容；GitHub/Hugging Face 用于核对当前开放代码、模型和数据状态。对于论文与 Hub 数据量不一致之处，报告分别列出，不做静默合并。